

```

In [1]: 1 # Import Packages
2
3 import numpy as np
4 import matplotlib.pyplot as plt
5 from scipy import stats as stats
6 from sklearn.linear_model import LinearRegression
7 from sklearn.decomposition import PCA
8 import scipy
9 import statsmodels.api as sm
10 import pandas as pd
11
12 from statsmodels.tsa.stattools import coint, grangercausalitytests
13 import matplotlib
14
15 from itertools import groupby, count
16 import pickle
17 import yfinance as yf
18
19 from dateutil.parser import parse
20 import datetime
21
22 from scipy.optimize import brentq
23 import traceback
24
25 import quantstats as qs
26 %matplotlib inline
27
28 # Allow Multiple Outputs in Cells
29 from IPython.core.interactiveshell import InteractiveShell
30 InteractiveShell.ast_node_interactivity = "all"
31
32 # Disable Warnings
33 import warnings
34 warnings.filterwarnings('ignore')
35

```

Principal Components by Hand

```

In [2]: 1 # Download Stocks
2 stocks = yf.download('SPY,QQQ', start='2020-1-1', progress=False).Close
3 stocks.head()

```

Out[2]:

	QQQ	SPY
Date		
2020-01-02	216.160004	324.869995
2020-01-03	214.179993	322.410004
2020-01-06	215.559998	323.640015
2020-01-07	215.529999	322.730011
2020-01-08	217.149994	324.450012

```
In [3]: 1 # Equity Curve: PROD(1+R)-1
2 stocks.pct_change().add(1).cumprod().add(-1).plot();
```



```
In [4]: 1 # Equity Curve of Centered Data i.e. with mean return removed
2 centered_returns = stocks.pct_change().dropna() - stocks.pct_change().n
3 centered_returns.add(1).cumprod().plot();
```



```
In [5]: 1 # Covariance of Centered Returns
2 S = centered_returns.cov()
3 print(S)
```

	QQQ	SPY
QQQ	0.000277	0.000213
SPY	0.000213	0.000191

```
In [6]: 1 # Calculate Eigenvalues and Eigenvectors of Covariance Matrix
        2 E = np.linalg.eig(S)
        3
        4 # EigenValues
        5 print('EigenValues')
        6 eValues=E[0]
        7 eValues
        8
        9 # Eigenvectors
       10 print('Eigenvectors - Each column is an Eigenvector, look closely they
       11 eVectors=E[1]
       12 eVectors
```

EigenValues

Out[6]: array([4.51142831e-04, 1.60230932e-05])

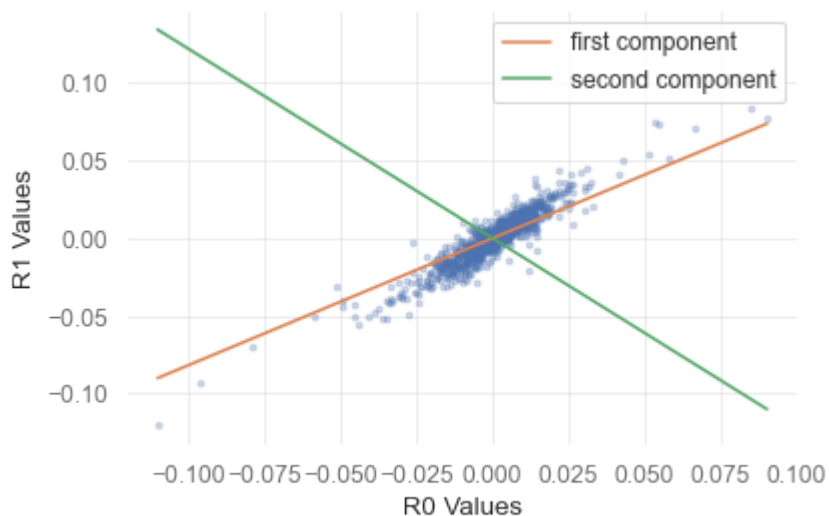
Eigenvectors - Each column is an Eigenvector, look closely they are perpendicular!

Out[6]: array([[0.77374516, -0.63349698],
 [0.63349698, 0.77374516]])

```

In [7]: 1 # Plot the Correlation of Stock Centered Returns
2
3 # 1. Compute Data for Principal Component Lines
4 x_axis = np.linspace(np.nanmin(centered_returns.SPY), np.nanmax(centered_returns.SPY), 100)
5
6 dx1 = eVectors[0,0]
7 dy1 = eVectors[1,0]
8 slope1 = dy1/dx1
9
10 dx2 = eVectors[0,1]
11 dy2 = eVectors[1,1]
12 slope2 = dy2/dx2
13
14 y_axis1 = slope1 * x_axis
15 y_axis2 = slope2 * x_axis
16
17 # 2. Plot Centered Return Correlation Between X and Y
18 plt.plot(centered_returns.SPY, centered_returns.QQQ, '.', alpha=0.25)
19
20 # 3. Plot Principal Components
21 plt.plot(x_axis, y_axis1, label='first component')
22 plt.plot(x_axis, y_axis2, label='second component')
23
24 plt.legend()
25
26 plt.xlabel('R0 Values')
27 plt.ylabel('R1 Values')
28 plt.show();

```



Principal Components and Explained Variance

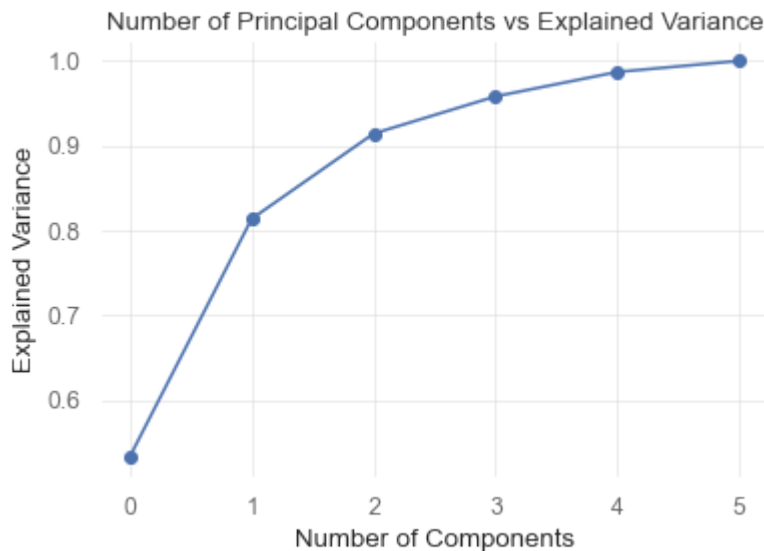
```

In [8]: 1 # Downloading More Instruments
2 equities = yf.download('GOOG,AMZN,NFLX,BAC,JPM,WFC', start='2020-1-1',

```

```
In [9]: 1 # Build a PCA Function that returns eigenvectors in order of their sign
2 def pca(assets):
3     x = assets.pct_change().dropna()
4     p = np.linalg.eig((x-x.mean()).cov())
5     evalues = p[0]
6     evectors = p[1]
7
8     # sort
9     return np.sort(evalues/sum(evalues))[:,::-1], evectors[:,np.argsort(e
```

```
In [10]: 1 # Explained Variance - How many principal components required to explain
2 evalues, evectors = pca(equities)
3
4 plt.plot(np.cumsum(evalues), '-o')
5 plt.title('Number of Principal Components vs Explained Variance')
6 plt.xlabel('Number of Components')
7 plt.ylabel('Explained Variance')
8 plt.show();
```



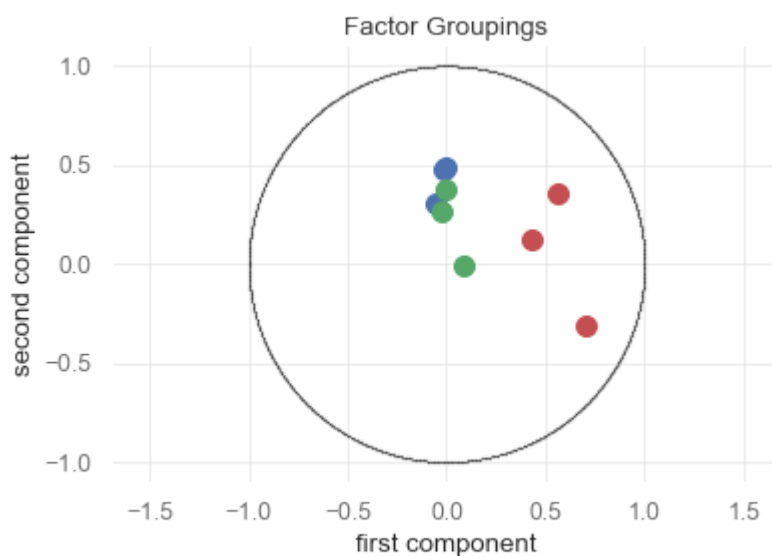
Factor Analysis

```
In [11]: 1 # Define our Assets
2 tech_stocks = yf.download('GOOG,AMZN,NFLX', start='2020-1-1', progress=
3 bank_stocks = yf.download('BAC,JPM,WFC', start='2020-1-1', progress=Fa
4 commodities = yf.download('CL=F,RB=F,GC=F', start='2020-1-1', progress=
```

```

In [12]: 1 # Factor Analysis
2 r = []
3 lookback = 50
4
5 r.append(tech_stocks['GOOG'].pct_change().iloc[-lookback:].values)
6 r.append(tech_stocks['AMZN'].pct_change().iloc[-lookback:].values)
7 r.append(tech_stocks['NFLX'].pct_change().iloc[-lookback:].values)
8
9 r.append(bank_stocks['BAC'].pct_change().iloc[-lookback:].values)
10 r.append(bank_stocks['JPM'].pct_change().iloc[-lookback:].values)
11 r.append(bank_stocks['WFC'].pct_change().iloc[-lookback:].values)
12
13 r.append(commodities['CL=F'].pct_change().iloc[-lookback:].values) # Cr
14 r.append(commodities['RB=F'].pct_change().iloc[-lookback:].values) # RE
15 r.append(commodities['GC=F'].pct_change().iloc[-lookback:].values) # Gc
16
17 R = np.array(r)
18
19 # Eigen System: Element [0]=eValues, [1]=eVectors
20 eigenSystem = np.linalg.eig(np.cov(R-np.mean(R)))
21
22 # Plot Results by Eigen Value Components
23 colour = ['r']*3 + ['b']*3 + ['g']*3
24
25 # Plot Assets
26 for j,i in enumerate(eigenSystem[1]):
27     plt.plot(i[0], i[1], '-o', c=colour[j], ms=10)
28
29 # Plot Circle
30 for i in np.linspace(-np.pi, np.pi, 1500):
31     plt.plot(np.cos(i), np.sin(i), 'k.', ms=0.3)
32
33 plt.title('Factor Groupings')
34 plt.xlabel('first component')
35 plt.ylabel('second component')
36 plt.axis('equal')
37 plt.show();

```



Factor Analysis Automation

```

In [13]: 1 def plot_assets_by_principal_components(dataFrame, symbols, lookback):
2         r = []
3
4         for symbol in symbols:
5             r.append(dataFrame[symbol].pct_change().iloc[-lookback:].values)
6
7         R = np.array(r)
8         eigenSystem = np.linalg.eig(np.cov(R - np.mean(R)))
9
10        for i, symbol in zip(eigenSystem[1], symbols):
11            plt.plot(i[0], i[1], '-o', ms=10)
12            # Adjust text position for better alignment
13            x_offset = 0.02 # Adjust based on preference
14            y_offset = 0.02 # Adjust based on preference
15            plt.text(i[0] + x_offset, i[1] + y_offset, symbol, fontsize=8,
16
17        # Plot Circle
18        for i in np.linspace(-np.pi, np.pi, 1500):
19            plt.plot(np.cos(i), np.sin(i), 'k.', ms=0.3)
20
21        plt.title('Factor Groupings')
22        plt.xlabel('First Component')
23        plt.ylabel('Second Component')
24        plt.axis('equal')
25        plt.show();

```

```

In [14]: 1 # Run Factor PCA for the below Assets
2         symbols = ['SPY', 'QQQ', 'GOOG', 'AMZN', 'MSFT', 'PYPL', 'CRM', 'BAC',
3         df = yf.download(symbols, start='2023-1-1', progress=False).Close
4
5         plot_assets_by_principal_components(df, symbols, 252)

```

